

5조 최종 보고서

Q-time과 예방보전 및 확률적 장비 고장을 고려한
반도체 공정 스케줄링 알고리즘 비교 연구박성현¹, 김형훈², 김진호³, 서훈⁴, 이승민⁵¹ 숭실대학교 산업정보시스템공학과 20232808² 숭실대학교 산업정보시스템공학과 20192208³ 숭실대학교 산업정보시스템공학과 20211275⁴ 숭실대학교 산업정보시스템공학과 20211286⁵ 숭실대학교 산업정보시스템공학과 20211296

Abstract 본 연구는 Q-time 제약, 확률적 장비 고장, 예방보전(PM)을 포함하는 반도체 공정 스케줄링 문제를 다룬다. SMT_2020 테스트베드를 기반으로 제품 경로, 장비 그룹, setup, Q-time, 고장 및 PM 파라미터를 재구성하고, SimPy 기반 이산 사건 시뮬레이션 환경에서 여러 스케줄링 접근법을 비교한다. 기본 dispatching rule, Composite rule, PM rule, Genetic Algorithm 기반 정책을 같은 입력 데이터와 시뮬레이터에서 평가하며, 성능은 makespan과 평균 Q-time violation을 함께 반영한 정규화 목적함수로 측정한다. 본 보고서는 각 접근법의 구현 방식과 평가 기준을 정리하고, 생산 효율성과 Q-time 제약 관리 사이의 trade-off를 분석한다. 실험 결과, Composite rule은 여러 priority term을 결합하는 rule-based 방식으로 단일 기준 rule들보다 makespan과 평균 Q-time violation의 균형이 좋았고, PM을 결합해도 이 균형을 유지하면서 처리 중 고장을 줄였다. 반면 GA는 작업 순서, 장비 배정, PM threshold를 함께 탐색하여 평균 Q-time violation을 더 낮췄지만, 계산 시간이 크게 길어 실시간 dispatching rule보다는 사전 최적화 또는 비교 기준에 가까웠다.

Keywords 반도체 공정, 스케줄링, Q-time, 예방보전, 장비 고장, Dispatching rule, Genetic Algorithm

1 서론

반도체 제조 공정에서는 여러 작업이 정해진 공정 순서를 따라 장비를 점유하며 처리된다. 제한된 장비를 어떤 작업에 어떤 순서로 할당하는지는 납기 준수, 장비 활용률, 생산 리드타임에 직접 영향을 주므로, 작업 순서와 장비 할당을 함께 결정하는 스케줄링 의사결정이 중요하다.

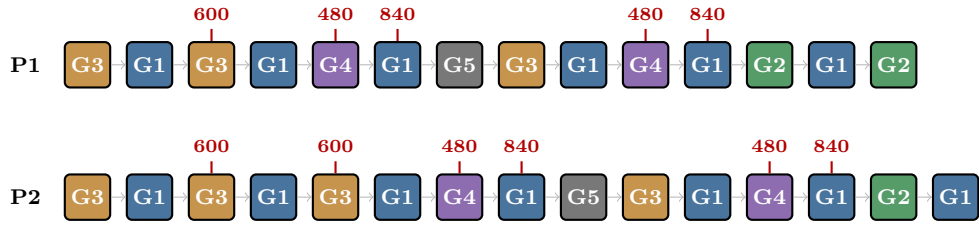
반도체 공정은 단순히 작업의 처리 순서를 정하는 문제로 보기 어렵다. 하나의 작업은 정해진 공정 순서를 따라 여러 장비를 거쳐 처리되며, 특정 공정은 복수의 대체 장비에서 수행될 수 있다. 또한 동일한 공정이라도 선택된 장비에 따라 처리 시간이 달라지므로, 작업 순서뿐 아니라 장비 할당까지 함께 결정해야 한다. 이러한 구조는 공정별 대체 장비 선택을 포함하는 Flexible Job Shop Scheduling Problem(FJSP)의 특성과 맞닿아 있다 (Brandimarte, 1993).

그러나 실제 제조 환경에서는 작업 순서와 장비 할당 외에도 추가로 고려해야 할 운영 특성이 존재한다. 반도체 공정에는 한 공정이 완료된 뒤 다음 공정이 일정 시간 안에 시작되어야 하는 Q-time 제약이 존재하며, 이를 초과하면 단순한 일정 지연을 넘어 품질 저하로 이어지고 위반 정도에 따라 재작업이나 폐기가 발생할 수 있다(Cho et al., 2014). 여기에 더해 장비는 예방보전(Preventive Maintenance, PM)으로 계획적으로 비가동되거나 예기치 못한

고장으로 중단될 수 있다 (Al-Hinai & ElMekkawy, 2011; Cassady & Kutanoglu, 2005). 따라서 반도체 공정 스케줄링은 Q-time과 같은 품질 제약, PM과 같은 계획된 비가동, 장비 고장이라는 확률적 불확실성이 동시에 작용하는 동적 의사결정 문제로 볼 수 있다.

이러한 환경에서는 전체 작업 완료 시간인 makespan만으로 스케줄링 성능을 평가하기 어렵다. 특정 작업을 빠르게 처리하더라도 다른 작업이 장시간 대기하면 Q-time violation이 증가할 수 있고, 반대로 Q-time violation을 줄이기 위해 특정 작업을 우선 처리하면 전체 완료 시간이 늘어날 수 있다. 그 결과 생산 효율성과 품질 제약 관리 사이의 균형이 필요하며, makespan과 Q-time violation을 함께 반영한 의사결정 기준이 요구된다.

이와 관련해 선행연구에서는 FJSP 구조, Q-time 제약, 예방보전, 장비 고장이 각각 중요한 스케줄링 요소로 다루어져 왔다 (Al-Hinai & ElMekkawy, 2011; Brandimarte, 1993; Cassady & Kutanoglu, 2005; Cho et al., 2014). 예를 들어 공정별 대체 장비 선택을 고려한 FJSP 연구가 제시되었고 (Brandimarte, 1993), 공정 간 대기시간 제약을 모델링한 연구와 생산 스케줄에 예방보전을 연계한 연구도 수행되었다 (Cassady & Kutanoglu, 2005; Cho et al., 2014). 또한 장비 고장이 존재하는 유연 작업장 스케줄링 문제도 다루어졌다 (Al-Hinai & ElMekkawy, 2011). 이들 연구는 본 연구의 문제 설정과 연결되는 주요 요소를



Colored boxes indicate machine groups G1–G5. Red labels = Q-time limit (min).

figure 2.1.1: 제품 타입별 공정 경로

제공하지만, Q-time violation, 장비 고장에 따른 가용성 변화, 예방보전 의사결정을 하나의 시뮬레이션 환경에서 함께 평가하는 구조와는 차이가 있다.

본 연구는 Q-time 제약, 확률적 장비 고장, PM을 하나의 시뮬레이션 환경에서 함께 고려한다. 기존 접근법으로는 rule-based algorithm을 구성하고, 더 넓은 해공간을 탐색하기 위해 Genetic Algorithm을 함께 적용한다 (Pezzella et al., 2008). 이후 두 접근법의 makespan과 Q-time violation을 비교하고, 생산 완료 시간과 공정 간 시간 제약 사이의 trade-off를 분석한다.

2 문제 설정 및 시뮬레이션 구성

2.1 데이터

본 연구는 반도체 제조 테스트베드 SMT_2020 (Semiconductor Manufacturing Testbed 2020)을 본 문제에 맞게 재구성한 데이터셋을 사용한다. 데이터셋은 60개의 작업과 60대의 장비로 구성되며, 작업은 30개씩 P1과 P2 두 제품 타입으로, 장비는 G1–G5 5개의 그룹으로 구분한다.

figure 2.1.1은 제품 타입별 공정 경로와 특정 공정의 Q-time 한도를 보여준다. P1은 14개, P2는 15개 공정으로 구성되며, 동일 장비 그룹을 여러 공정 단계에서 반복 처리 받는다. 각 공정은 지정된 장비 그룹 내 어느 장비에서나 처리받을 수 있다.

table 2.1.1은 장비 그룹별 입력 파라미터를 보여준다. 처리시간은 제품 타입별로 서로 다른 균등분포에서 추출된 시간을 따르며, setup은 장비가 다른 제품 타입으로 전환하여 공정을 처리하는 경우만 발생한다. PM 시간은 사후 수리시간보다 짧은 값을 가진다.

2.2 목적 함수

성능 기준은 makespan과 평균 Q-time violation을 함께 반영한 정규화 목적함수로 정의한다. 시뮬레이션 환경의 운영 제약을 만족하는 모든 스케줄 $\sigma \in \Sigma$ 가운데 다음 목적함수를 최소화하는 해를 찾는 알고리즘을 설계한다.

$$\min_{\sigma \in \Sigma} J(\sigma) = w_1 \frac{C_{\max}}{mk_{\text{ref}}} + w_2 \frac{AQT}{qv_{\text{ref}}}$$

$$C_{\max} = \max_{i \in \mathcal{J}} C_i, \quad AQT = \frac{1}{N} \sum_{i=1}^N v_i$$

$\sigma \in \Sigma$ 운영 제약을 만족하는 스케줄

C_i 작업 i 의 완료시간

v_i 작업 i 의 Q-time violation 총량 (분)

N 작업 수 (= 60)

$mk_{\text{ref}}, qv_{\text{ref}}$ 정규화 기준값 (baseline 관측 최댓값)

w_1, w_2 가중치 ($w_1 = w_2 = 1$)

makespan과 AQT 는 단위는 같지만 관측 범위가 달라, 정규화 없이 더하면 값의 규모가 큰 makespan이 J 를 지배한다. 이를 막기 위해 기본 dispatching rule을 동일 seed로 반복 실행했을 때 관측된 최댓값($mk_{\text{ref}} \approx 14,734$ 분, $qv_{\text{ref}} \approx 1,140$ 분)을 기준값으로 두어 각 항을 baseline 대비 비율로 환산하며, 가중치는 두 지표를 동등하게 두었다.

table 2.1.1: 장비 그룹별 입력 파라미터

그룹	장비 수	처리시간 범위 (분)		전환 setup (분)	고장분포 파라미터		PM 시간 (분)	수리시간 (분)
		P1	P2		형상 k	척도 λ (분)		
G1	30	$U(440, 520)$	$U(470, 490)$	120	3.0	11000	80	220
G2	4	$U(170, 190)$	$U(158, 162)$	40	2.8	11000	200	600
G3	15	$U(440, 460)$	$U(390, 410)$	120	2.5	11000	50	150
G4	8	$U(325, 375)$	$U(325, 375)$	90	2.2	11000	200	700
G5	3	$U(145, 155)$	$U(145, 155)$	50	3.3	11000	70	200

2.3 시뮬레이션 환경

본 시뮬레이션은 Python의 SimPy 라이브러리를 기반으로 한 이산 사건 시뮬레이션(discrete-event simulation)으로 구현되었다. 본 절에서는 입력 데이터가 시뮬레이션 안에서 어떤 상태 변화와 사건으로 처리되는지, 그리고 그 결과로 makespan과 Q-time violation이 어떻게 산출되는지 설명한다. 서술 순서는 작업 관점의 공정 흐름, 장비 상태 변화와 고장·수리·PM 처리, Q-time violation 산출이다.

2.3.1 작업 공정 흐름

figure 2.3.1은 작업의 공정 흐름을 보여준다. 모든 작업은 시점 0에 투입되고, 동일한 작업 우선순위를 가지며, 장비 간 이동 시간은 0으로 가정한다. 작업 할당 전 항상 대기 공간인 스톡ер(stocker)를 거치며, 스톡ер의 용량은 무한대로 가정한다. 작업이 다음 공정으로 진행할 수 있는 상태가 되었더라도 처리 가능한 장비가 즉시 사용 가능하지 않으면 작업은 스톡ер에서 대기한다.

장비가 사용 가능해지면 대기 작업과 장비의 조건을 바탕으로 dispatching이 수행된다. dispatching의 진입 시점은 두 가지로 구분된다. (i) 작업이 스톡ер에 도착했을 때 처리 가능한 장비가 사용 가능하면 즉시 할당하고, (ii) 장비가 유휴 상태가 되었을 때 대기 작업이 있으면 후술할 dispatching 알고리즘에 따라 작업을 선택한다.

작업이 장비에 할당되면 장비에서 직전에 처리한 작업의 유형과 현재 작업의 유형을 비교하여 setup 필요 여부를 판단한다. setup이 필요한 경우 setup을 먼저 수행한 뒤 가공을 시작하며, setup이 필요하지 않은 경우에는 바로 가공을 시작한다. setup 도중 혹은 가공 도중 장비 고장이 발생할 수 있으며, 고장 시 해당 공정은 완료되지 않은 것으로 처리된다. 그 결과 중단된 작업은 같은 공정을 다시 수행해야 하는 상태로 스톡ер에 복귀하고, 이후 재할당 과정을 거친다. 고장으로 인한 중단과 재투입은 여러 회 반복될 수 있다.

모든 공정을 완료한 작업은 완료 상태로 간주되어 시뮬레이션에서 제외되며, 모든 작업이 완료되면 시뮬레이션이 종료된다. 이때 마지막 작업이 완료되어 시뮬레이션이 종료되는 시점을 makespan으로 기록한다.

2.3.2 장비 상태 변화

figure 2.3.2는 장비 상태가 시간에 따라 어떻게 변하는지 예시 간트차트를 통해 보여준다. 장비는 유휴, setup, 가공, 수리, PM 수행 중 하나의 상태를 가지며, 한 번에 하나의 작업만 처리한다.

각 장비의 고장 발생 시점은 장비별 고장 분포(Weibull)에서 추출한 다음 고장까지의 시간으로 결정한다. 해당 시간이 경과하면 장비는 고장 상태로 전환되며, 이는 유휴 상태뿐 아니라 setup 또는 가공 중에도 발생할 수 있다. 처리 중 고장이 발생하면 진행 중이던 공정은 중단되고, 장비는 정해진 수리 시간 동안 수리 상태를 거친 뒤 다시 사용 가능 상태로 복귀한다. 수리가 완료된 장비는 setup 판단

기준이 초기화되므로, 복귀 직후 처음 처리하는 작업에는 setup 시간이 발생하지 않는다.

예방보전(PM)은 후술할 PM rule에 따라 장비 고장 가능성을 낮추기 위해 수행한다. PM은 장비가 유휴 상태일 때만 수행하며, PM 예정 시점에 작업이 진행 중이면 해당 공정이 완료될 때까지 대기한 뒤 수행한다. PM 수행 중에는 고장이 발생하지 않는 것으로 간주한다. PM이 완료되면 장비는 사용 가능 상태로 복귀하며, 수리 직후와 동일하게 다음 고장 시점과 setup 판단 기준이 초기화되어 복귀 후 첫 작업에는 setup이 발생하지 않는다.

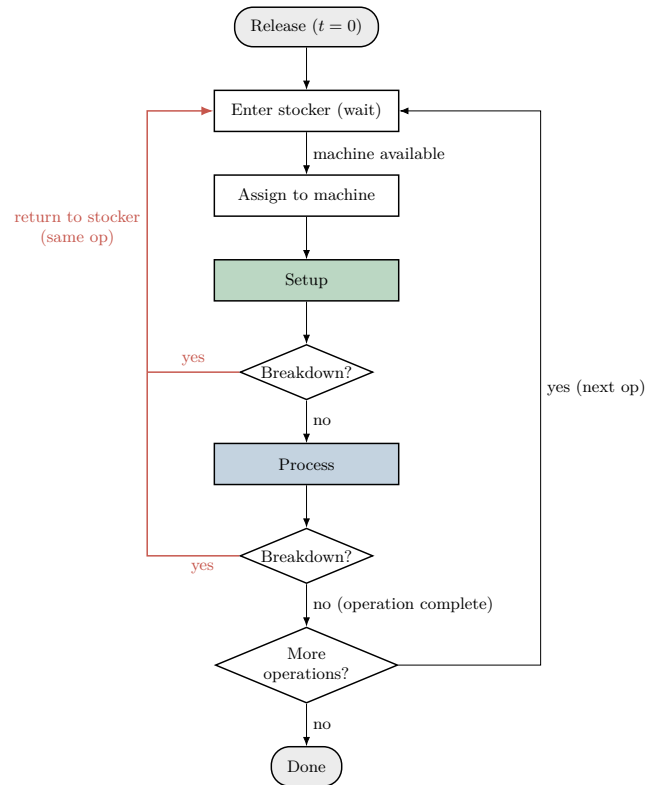


figure 2.3.1: 작업 흐름도

2.3.3 Q-time violation

figure 2.3.3은 Q-time 누적 규칙과 Q-time violation 산출 방식을 보여준다. Q-time은 한 공정이 완료된 후 다음 공정의 정상 가공이 시작되기 전까지 허용되는 최대 대기시간으로, 작업이 이 한도를 초과하면 Q-time violation이 발생한다.

Q-time은 이전 공정 완료 후 다음 공정의 정상 가공이 시작되기 전까지 누적되며, 스톡ер 대기과 setup뿐 아니라 고장으로 중단되어 완료되지 못한 가공 시간까지 포함한다.

작업이 Q-time 한도를 초과하면 그 초과 시간이 Q-time violation 시간으로 기록되며, 시뮬레이션 종료 시에는 총 Q-time violation 시간을 전체 작업 수로 나눈 평균값을 산출한다.

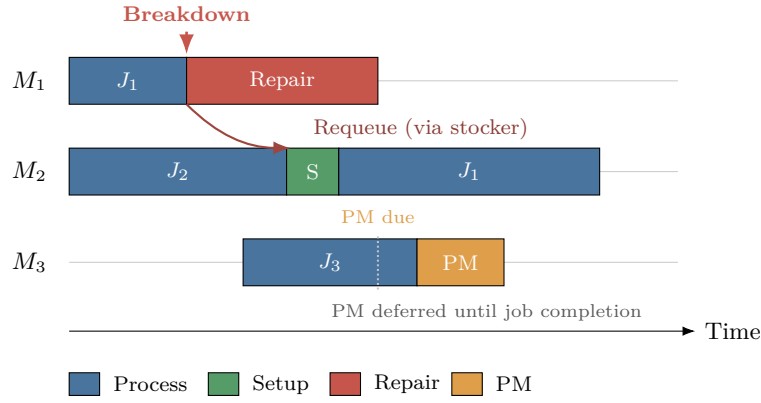


figure 2.3.2: 예시 간트 차트

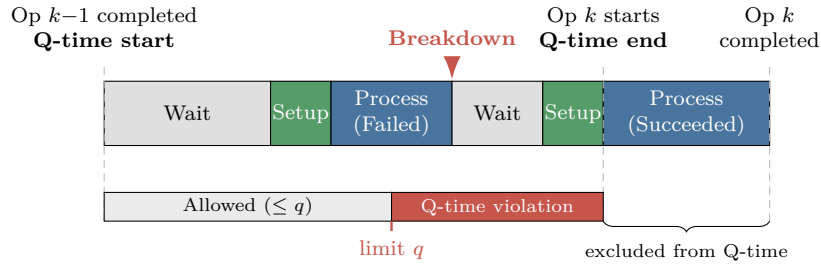


figure 2.3.3: Q-time 누적 규칙

2.4 Rule-based algorithm

본 절은 dispatching rule 기반 스케줄링을 다룬다. 단일 기준으로 작업을 선택하는 기본 dispatching rule 6종을 baseline으로 정의하고, 이어서 여러 기준을 결합한 Composite rule을 제시한다. 두 방법 모두 장비가 유헤 상태가 되는 순간 stocker의 후보 작업 중 하나를 고르는 규칙으로, 전체 스케줄을 사전에 고정하지 않고 시뮬레이션 진행 중 실시간으로 작업을 선택한다.

2.4.1 기본 dispatching rule

table 2.4.1은 본 연구에서 사용되는 6개의 기본 dispatching rule과 각 rule이 작업을 선택하는 기준을 보여준다. 각 rule은 현재 공정을 처리할 수 있는 후보 작업 중 하나를 단일 기준으로 선택한다. 이 6종은 이후 Composite rule과 GA가 처리시간·setup·Q-time을 결합했을 때의 성능 차이를 가늠하는 비교 기준이 된다

table 2.4.1: 기본 dispatching rule과 선택 기준

Rule	선택 작업 j^*
random	$j^* \sim \text{Uniform}$
FIFO	$\arg \min_j a_j$
SPT	$\arg \min_j p_{jm}$
LPT	$\arg \max_j p_{jm}$
MIN_QTIME	$\arg \min_j r_j(t)$
SPTSSU	$\arg \min_j (s_{jm} + p_{jm})$

a_j : stocker 도착시각

p_{jm} : 장비 m 에서의 처리시간

$r_j(t)$: 잔여 Q-time (초과 시 음수)

s_{jm} : setup 시간

각 dispatching rule은 서로 다른 정보를 기준으로 작업 우선순위를 결정한다. SPT와 LPT는 처리시간, MIN_QTIME은 잔여 Q-time, SPTSSU는 setup을 포함한 처리시간처럼 하나의 주요 기준에 따라 작업 우선순위를 정한다. 이처럼 각 rule은 특정 관점에서는 의미 있는 판단 기준을 제공하지만, 한 번의 dispatching decision에서 여러 운영 요소를 동시에 고려하지는 못한다.

2.4.2 Composite rule

Composite rule은 후보 작업을 하나의 기준으로만 정렬하지 않고, 처리시간, Q-time 여유, setup 부담, 공정 진행 상태를 하나의 점수로 묶어 비교한다. 후보 작업 j 에 대해 각 우선순위 항 $T_k(j)$ 를 산출한 뒤, 항별 경험적 최댓값 $T_k^{\max}$ 로 나누어

정규화한다($\hat{T}_k(j) = T_k(j)/T_k^{\max}$). 모든 항은 값이 작을수록 먼저 선택되도록 부호를 맞춘다. 최종 점수는 다음과 같다.

$$\pi(j) = \sum_{k \in S} w_k \hat{T}_k(j), \quad j^* = \arg \min_j \pi(j)$$

여기서 S 는 Composite rule에 포함되는 항 집합, w_k 는 각 항의 가중치이다. 후보 항은 서로 다른 스케줄링 측면을 반영하는 6개로, table 2.4.2와 같다.

table 2.4.2: Composite rule 우선순위 항

항	의미
PT	현재 장비에서의 처리시간
SLACK	남은 Q-time 여유 (제약 없으면 후순위)
SETUP	제품 타입 전환 setup 시간
C_TRANSITION	다음 공정그룹 전환의 Q-time 위험도
COMPLETION_FAST	공정 진행률 (높을수록 우선)
WAITING	누적 대기시간 (길수록 우선)

가중치는 직접 지정하지 않고 탐색한다. 6개 항을 모두 후보로 두고, 가중치 합이 1이 되는 simplex 위에서 Particle Swarm Optimization(PSO)으로 목적함수 J 가 낮아지는 가중치 벡터를 탐색한다. 탐색 단계에서 각 후보 가중치는 여러 seed의 반복 시뮬레이션 평균 J 로 평가하며, 선택된 가중치의 최종 성능은 성능 평가 방법 절의 반복 실행 기준에 따라 비교한다.

전체 가중치 탐색 절차는 Algorithm 1과 같다.

Algorithm 1. Composite dispatching-rule weight optimization with simplex-PSO

Input: $N, T_{MAX}, D, \omega, c_1, c_2$

Output: w^*

```

1 Initialization:
2 for each particle  $i = 1, \dots, N$  do
3   Sample position  $x_i \sim U([0, 1])$ 
4   Initialize velocity  $v_i \leftarrow 0$ 
5 end for
6 for each particle  $i = 1, \dots, N$  do
7    $pbest_i \leftarrow x_i, F(pbest_i) \leftarrow F(x_i)$ 
8 end for
9  $gbest \leftarrow \arg \min_i F(pbest_i)$ 
10  $t \leftarrow 1$ 
11 Update:
12 while  $t \leq T_{MAX}$  do
13   for each particle  $i = 1, \dots, N$  do
14     Pick random numbers:  $r_1, r_2 \sim U(0, 1)^D$ 
15     Update velocity  $v_i \leftarrow \omega v_i + c_1 r_1 (pbest_i - x_i) + c_2 r_2 (gbest - x_i)$ 
16     Update position  $x_i \leftarrow x_i + v_i$ 
17     Project onto simplex
18      $x_i \leftarrow \max(x_i, 0) / \sum_d \max(x_{i,d}, 0)$ 
19   end for
20   for each particle  $i = 1, \dots, N$  do
21     If  $F(x_i) < F(pbest_i)$  then  $pbest_i \leftarrow x_i, F(pbest_i) \leftarrow F(x_i)$ 
22   end for
23    $gbest \leftarrow \arg \min_i F(pbest_i)$ 
24    $t \leftarrow t + 1$ 
25 end while
26 return  $w^* \leftarrow gbest$ 

```

가중치 탐색은 PM을 비활성화하고 고장은 활성화한 조건에서 수행하였다. 따라서 Composite rule의 가중치는 dispatching 우선순위에 맞춰 조정된 값이며, PM과 결합했을 때의 효과는 별도 실험에서 확인한다.

2.5 PM rule

PM rule은 각 장비를 언제 선제 점검할지를 결정한다. 본 연구는 threshold rule을 채택하고, age replacement rule을 비교 기준으로 함께 본다. 두 rule 모두 장비의 누적 고장확률을 Weibull 분포로 모델링하며, 마지막 PM 또는 수리 이후 경과시간 t 에 대해

$$F(t) = 1 - \exp\left(-\left(\frac{t}{\lambda}\right)^k\right)$$

로 정의한다. 여기서 k 는 형상 모수, λ 는 척도 모수이며, 본 실험에서는 전 장비에 $\lambda = 11,000$ 분을 적용하고 형상 모수는 그룹별로 $k = 3.0, 2.8, 2.5, 2.2, 3.3$ 으로 둔다.

2.5.1 Threshold rule

$$t_{PM} = \lambda (-\ln(1 - \theta))^{1/k}$$

Threshold rule은 장비의 누적 고장확률 $F(t)$ 가 사전에 정한 임계값 θ 에 도달하면 PM을 수행하는 방식이다. θ 가 작을수록 PM이 이르게 발생하고, θ 가 클수록 장비를 더 오래 사용한 뒤 PM이 수행된다. 따라서 Threshold rule은 고장 위험을 어느 수준까지 허용할 것인지를 직접 조정하는 PM rule로 볼 수 있다.

2.5.2 Age Replacement rule

$$g(t) = \frac{t_p [1 - F(t)] + t_f F(t)}{\int_0^t [1 - F(u)] du}$$

$g(t)$ PM 주기 t 에서의 단위 운전시간당 기대 비가동시간
 t 마지막 PM 또는 수리 이후의 경과시간이자 후보 PM 주기
 t_p PM 수행시간
 t_f 고장 후 수리 시간
 $F(t)$ 경과시간 t 까지의 누적 고장확률
 $\int_0^t [1 - F(u)] du$ PM 또는 고장 전까지의 기대 운전시간

Age Replacement rule은 연령 교체 모델 (Barlow & Proschan, 1965)에 기반하여, $g(t)$ 가 가장 작아지는 t 를 PM 주기로 선택한다. Threshold rule이 누적 고장확률의 임계값을 기준으로 PM 시점을 정하는 반면, Age Replacement rule은 PM으로 인한 계획 비가동과 고장 후 수리 손실의 균형을 기준으로 한다.

2.6 유전 알고리즘 (GA)

본 절에서는 서론에서 제시한 GA 기반 접근의 구체적인 설계를 기술한다. 제조 공정 스케줄링에서 GA는 작업 순서와 자원 할당을 하나의 해 표현으로 함께 탐색하는 방식으로 활용되어 왔으며 (Kim et al., 2021; Pezzella et al., 2008), 반도체 Q-time 제약 스케줄링에서는 염색체 인코딩과 디코딩 절차의 설계가 중요한 요소로 다루어진다 (Park & Huh, 2026). 본 연구의 GA는 작업 투입 순서, 공정별 장비 배정, 장비별 예방보전(PM) 기준을 하나의 염색체로 표현하고, 선택, 교차, 돌연변이를 반복하여 더 나은 조합을 탐색한다.

2.6.1 염색체 인코딩

본 연구의 염색체는 job_seq, machine, pm의 세 부분으로 구성된다. job_seq는 작업 투입 순서를 나타내는 순열이며, 선택 가능한 작업이 여러 개일 때 해당 순서에 따라 작업을 선택한다. machine은 각 공정이 수행될 장비를 처리 가능 장비 목록의 인덱스로 나타낸다. 따라서 하나의 공정이 여러 장비 중 하나에 배정될 수 있는 구조를 염색체에 반영할 수 있다. pm은 장비별 예방보전 기준을 사전에 정의한 후보값 중 하나로 지정한다. 본 연구에서는 PM 기준 후보를 {0.05, 0.10, 0.20, 0.35, 0.50, 0.70, 0.90, 1.00}으로 두었으며, 각 값은 장비의 Weibull 누적고장확률이 해당 수준에 도달했을 때 PM을 수행하는 기준으로 사용된다.

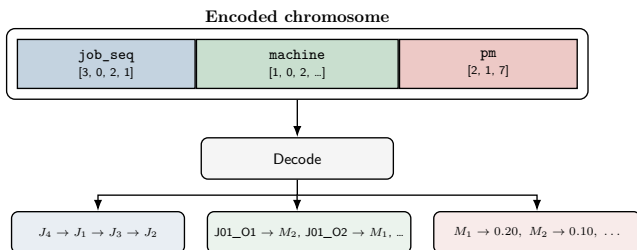


figure 2.6.1: GA 염색체 인코딩 및 디코딩 예시

figure 2.6.1은 정수 인덱스로 표현된 염색체가 디코딩을 거쳐 작업 투입 순서, 공정별 장비 배정, 장비별 PM 기준으로 변환되는 예시를 보인다. 장비 고장은 의사결정 변수로 인코딩하지 않고, 시뮬레이션 평가 과정의 상태 변화로 반영한다.

2.6.2 평가 및 세대 갱신

figure 2.6.2는 GA의 세대 갱신 과정을 나타낸다. 초기 집단은 무작위로 생성한다. job_seq는 모든 작업이 한 번씩 등장하는 순열로 생성하고, machine은 각 공정의 처리 가능 장비 중 하나를 선택하며, pm은 장비별 PM 기준 후보 중 하나를 선택한다. 생성된 염색체는 디코딩 후 시뮬레이션으로 평가하고, 그 결과를 바탕으로 해의 우열을 판단한다.

각 세대에서는 평가 결과가 우수한 상위 일부 염색체를 엘리트로 보존하여 다음 세대로 그대로 전달한다. 나머지 개체는 토너먼트

선택을 통해 부모 염색체를 선택한 뒤, 교차와 돌연변이를 거쳐 새로운 자식 염색체로 생성한다. 이후 엘리트와 자식 염색체를 합쳐 다음 세대 집단을 구성하고, 지정한 세대 수에 도달할 때까지 같은 과정을 반복한다. 이 방식은 현재까지 발견된 우수한 해를 보존하면서도 새로운 조합을 탐색하도록 한다.

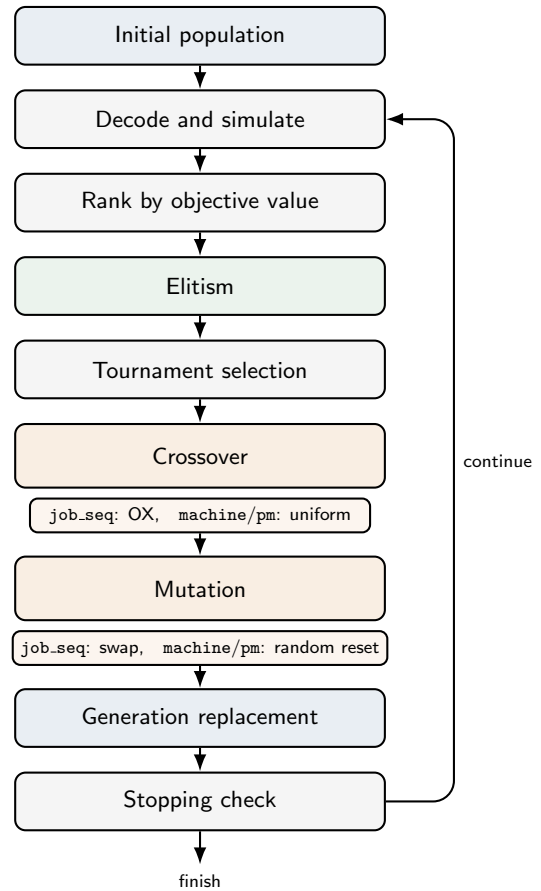


figure 2.6.2: GA 세대 갱신 흐름

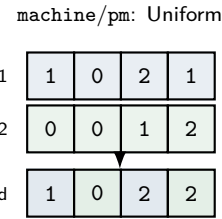
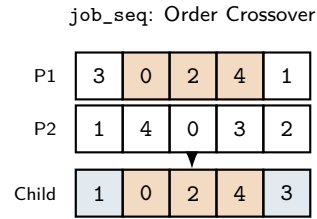
2.6.3 교차와 돌연변이

교차와 돌연변이는 염색체의 세 부분이 가진 구조를 유지하도록 다르게 적용한다. figure 2.6.3와 같이 job_seq는 순열 구조를 가져야 하므로 order crossover를 사용하여 작업의 중복이나 누락이 발생하지 않도록 한다. machine과 pm은 각 위치의 값이 독립적인 인덱스이므로 uniform crossover를 적용한다. 돌연변이의 경우 job_seq에는 두 위치의 값을 바꾸는 swap mutation을 적용하고, machine과 pm에는 가능한 후보 중 기존 값과 다른 값으로 바꾸는 random reset 방식을 적용한다. 이러한 연산을 통해 모든 세대의 염색체가 실행 가능한 시뮬레이션 입력으로 유지되도록 하였다.

2.7 성능 평가 방법

최종 성능 비교는 서로 다른 seed에서 30회 반복 독립 실행한 결과의 표본평균을 사용한다. Composite rule의 가중치 탐색

Crossover



Mutation

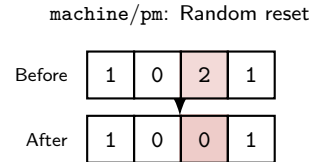
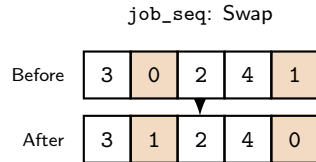


figure 2.6.3: 교차·돌연변이 연산 예시

단계에서는 계산 비용을 고려하여 각 후보 가중치를 10회 반복 시뮬레이션의 평균 J 로 평가한다.

정책 간 비교 기준은 목적 함수 절에서 정의한 정규화 목적함수 J 를 사용하고, PM의 효과를 해석하기 위한 보조 지표로 처리 중 고장 횟수를 함께 제시한다.

3 결과 및 분석

본 절에서는 Composite rule의 가중치 탐색 결과를 정리하고, PM을 적용하지 않은 조건에서 기본 dispatching rule들과 비교한다. 이어서 Composite rule에 PM을 적용한 효과와 GA의 탐색 결과를 살펴본 뒤, rule-based 접근인 Composite+PM과 GA를 비교한다.

3.1 Composite rule 가중치

table 3.1.1: PSO 탐색으로 산출한 priority term 가중치

Term	Weight
PT	0.000
SLACK	0.432
SETUP	0.408
C_TRANSITION	0.065
COMPLETION_FAST	0.043
WAITING	0.052

SLACK과 SETUP의 가중치 합이 약 0.84로, 본 환경에서는 Q-time 여유와 setup 시간이 dispatching 성능을 좌우하는 핵심 항임을 보인다. PT의 가중치는 0으로 나타났는데, 이는 처리시간 단독 기준보다 setup과 Q-time slack을 함께 고려하는 편이 목적함수 개선에 더 직접적으로 기여했음을 의미한다. 나머지 항(C_TRANSITION, COMPLETION_FAST, WAITING)은 보조적으로 작동한다. PSO는 swarm 24, iteration 40으로 설정하였고, 각

후보 가중치는 탐색 단계에서 10회 반복 시뮬레이션의 평균 J 로 평가하였다.

3.2 기본 dispatching rule과 Composite rule 비교 (PM 미적용)

table 3.2.1은 PM을 적용하지 않은 조건에서 작업 선택 기준 자체가 성능에 미치는 영향을 보여준다.

table 3.2.1: 기본 dispatching rule과 Composite rule 성능 (PM 미적용)

Rule	Makespan (hour)	AQT (hour)	J
RANDOM	222.30	10.261	1.4518
FIFO	214.73	11.662	1.4933
SPT	216.94	8.325	1.3291
LPT	218.90	4.274	1.1268
MIN_QTIME	211.15	1.947	0.9739
SPTSSU	215.77	14.185	1.6286
COMPOSITE	210.49	1.232	0.9223

단일 rule은 강조하는 기준에 따라 성능이 갈렸다. MIN_QTIME은 Q-time violation을 직접 줄여 단일 rule 중 가장 낮은 J (0.974)를 보였고, SPTSSU는 setup을 고려하지만 Q-time slack을 반영하지 못해 AQT가 가장 컸다. Composite rule은 makespan 210.49, AQT 1.232로 J 0.9223을 기록하여 모든 단일 rule보다 낮았다. 이는 Q-time slack과 setup을 동시에 반영하고 transition risk와 waiting을 보조적으로 결합한 결과로 해석된다. 이에 따라 이후 PM 실험에서는 Composite rule을 기본 dispatching rule로 사용한다.

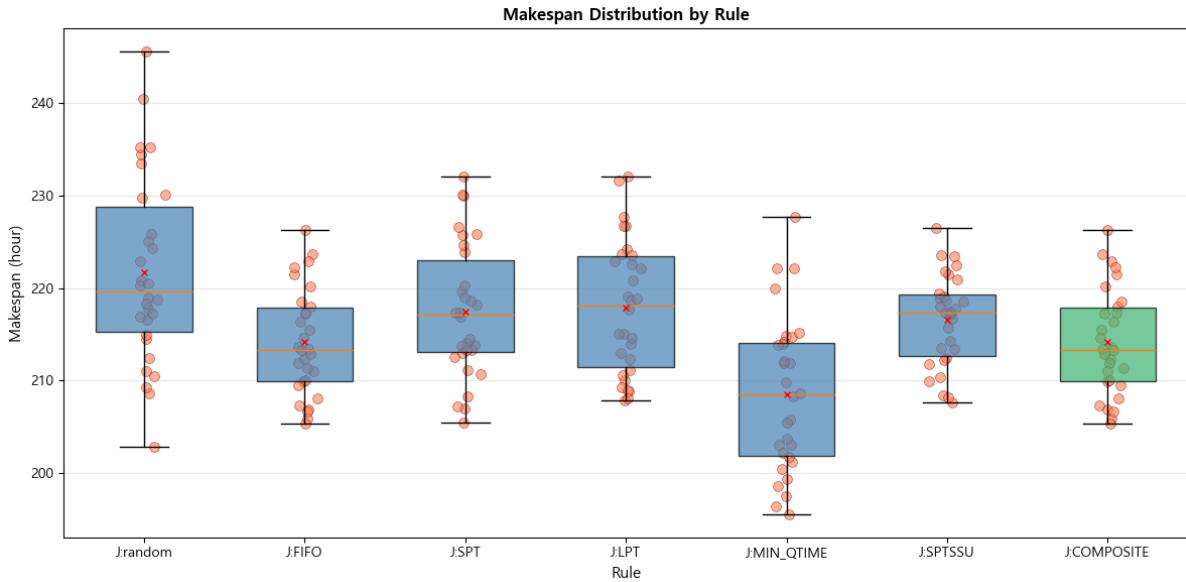


figure 3.2.1: Rule별 makespan 분포 (PM 미적용)

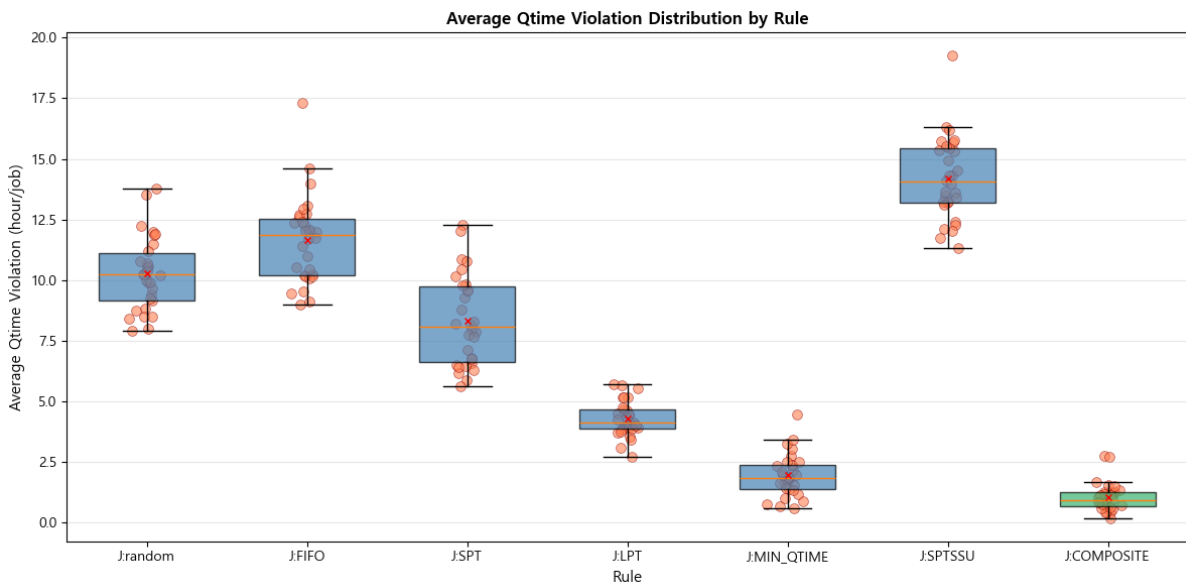


figure 3.2.2: Rule별 Q-time violation 분포 (PM 미적용)

3.3 Composite rule에 PM 적용

PM 미적용 비교에서 가장 낮은 J 를 보인 Composite rule에 PM threshold rule을 적용하고, threshold를 변화시키며 makespan, AQT, 처리 중 고장 횟수, J 를 비교하였다.

table 3.3.1: PM threshold에 따른 성능 (Composite)

PM threshold	Makespan (hour)	AQT (hour)	처리 중 고장	J
0.05	301.586	5.184	5.70	1.501
0.10	250.285	1.240	7.67	1.084
0.15	235.357	1.385	9.40	1.031
0.30	218.242	1.189	14.20	0.951
0.50	210.191	1.205	18.77	0.919

table 3.3.2: PM 정책별 성능 비교 (Composite)

정책	Makespan (hour)	AQT (hour)	처리 중 고장	J
PM off	210.49	1.232	20.43	0.9223
PM threshold	210.19	1.205	18.77	0.9192
PM age	220.56	1.248	13.20	0.9641

threshold가 낮으면(0.05) PM이 잦아 계획 비가동이 늘고 makespan이 301까지 커진다. 반대로 높이면 PM 횟수는 줄지만 처리 중 고장이 5.70에서 18.77까지 증가한다. J 기준으로는 threshold 0.5에서 0.919로 가장 낮았다. 정책 간 비교에서 threshold는 PM off와 makespan· J 가 비슷하면서 고장 횟수를 줄였고, age 정책은 고장을 더 줄이는 대신 makespan이 늘어 J 가 높았다. 즉 PM rule은 고장 감소와 계획 비가동 사이의 trade-off를 조정한다.

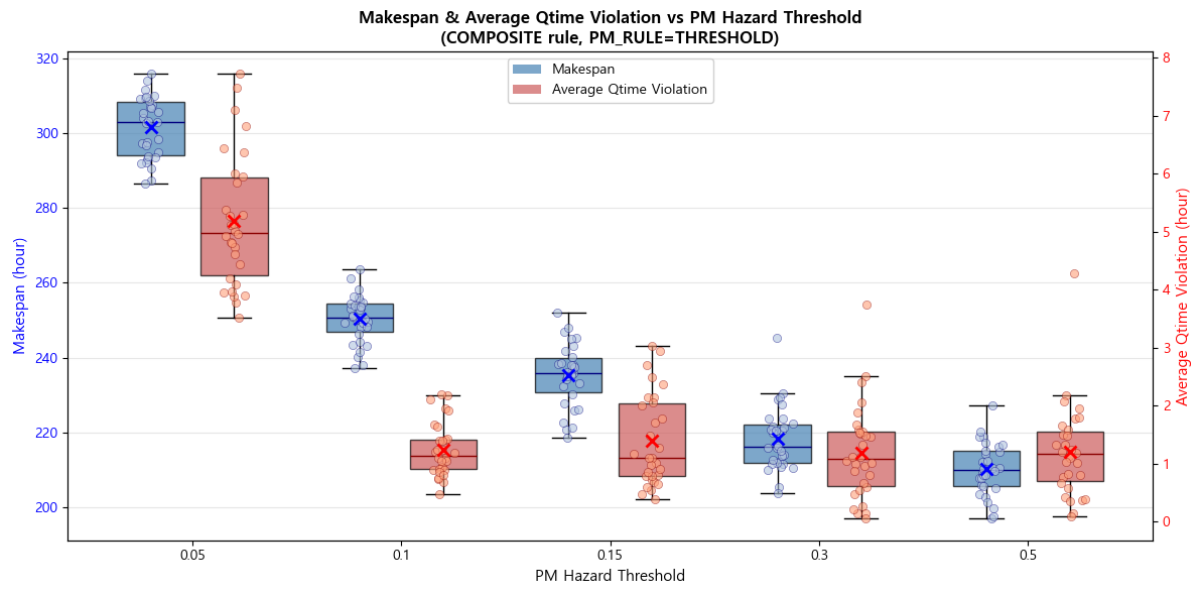


figure 3.3.1: PM threshold에 따른 makespan과 평균 Q-time violation (Composite, threshold rule)

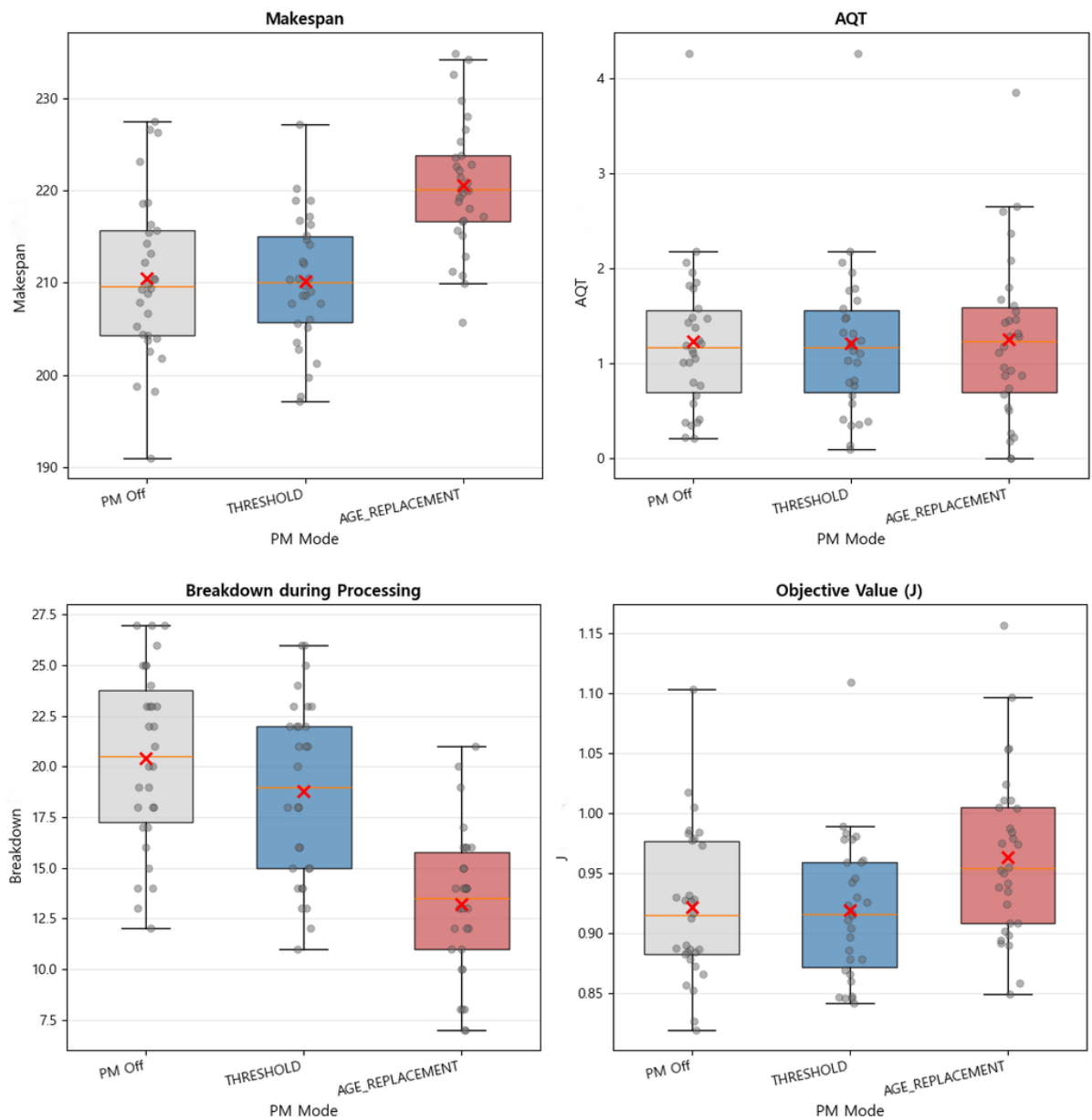


figure 3.3.2: PM 정책별 성능 분포 (Composite, PM off/threshold/age)

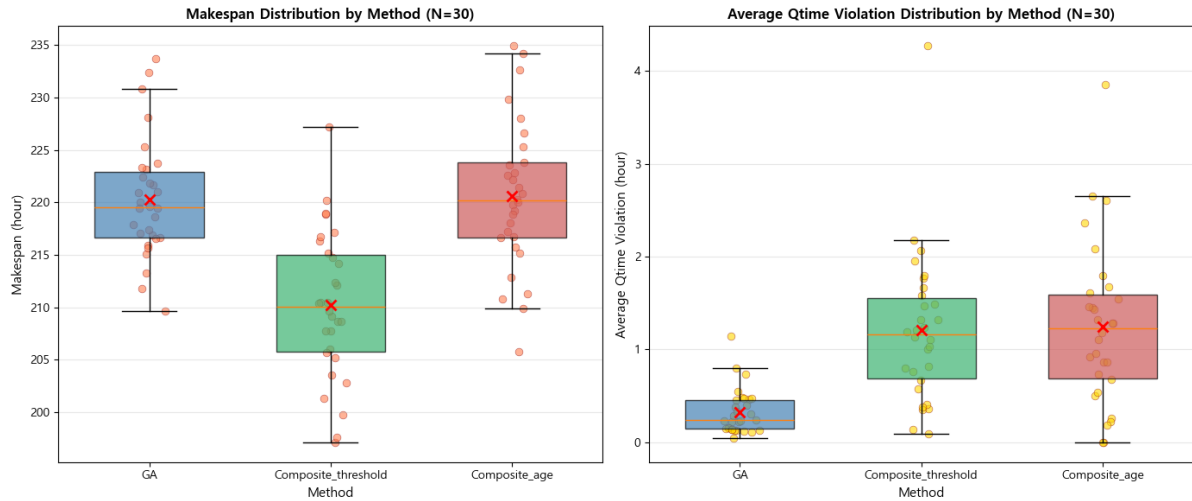


figure 3.5.1: 방법별 makespan 및 평균 Q-time violation 분포 (GA vs Composite+PM)

3.4 GA 결과

table 3.4.1: GA 실행 파라미터

Parameter	Value
Population size	100
Generations	1000
Crossover rate	0.8
Mutation rate (job_seq)	0.01
Mutation rate (machine)	0.0005
Mutation rate (PM)	0.005
Selection	Tournament ($k = 3$)
Elitism	1 chromosome

table 3.4.2: GA 30회 독립 실행 통계

지표	평균	표준편차	최소	최대
Best makespan (hour)	220.292	5.672	209.666	233.677
Best AQT (hour)	0.328	0.243	0.048	1.141
Best J	0.914	0.029	0.860	1.000
처리 중 고장 횟수	8.100	2.578	3.000	14.000

GA는 table 3.4.1의 설정을 고정한 뒤, seed를 달리하여 30회 독립 실행하였다. 각 실행에서는 최종 세대까지 탐색한 결과 중 목적함수 값이 가장 낮은 염색체를 해당 seed의 대표 해로 사용하였다. table 3.4.2는 이렇게 얻은 30개 대표 해의 성능 분포를 정리한 것이다.

30회 실행의 평균 J 는 0.914였으며, 최저값은 0.860이었다. 평균 makespan은 220.292 hour로 Composite rule보다 짧지는 않았지만, 평균 AQT는 0.328 hour로 낮게 나타났다. 즉 GA는 생산 완료시간을 줄이기보다는 Q-time violation을 완화하는 쪽에서 상대적으로 강점을 보였다. 이는 작업 순서, 장비 배치, PM threshold를 하나의 염색체 안에서 함께 조정했기 때문으로 볼 수 있다. 다만 AQT가 0.048–1.141 hour, 처리 중 고장이 3–14회로 벌어져 seed와 확률적 고장 조건에 따른 변동도 남아 있었다.

3.5 Composite+PM과 GA 비교

마지막으로 Composite rule에 PM을 결합한 rule-based 접근과 GA를 비교하였다(table 3.5.1).

table 3.5.1: Composite+PM과 GA 최종 비교

방법	Makespan (hour)	AQT (hour)	처리 중 고장	J	실행시간 (hour)
C+PM(threshold)	210.19	1.2046	18.7	0.9193	0.0108
C+PM(age)	220.56	1.2479	13.2	0.9638	0.0110
GA	220.29	0.3280	8.1	0.9143	365.68

GA는 J 0.914로 Composite+PM(threshold)의 0.919보다 약간 낮았고, 특히 AQT를 0.328 hour까지 줄였다. 작업 순서, 장비

배정, PM threshold를 함께 탐색했기 때문으로 보인다. 다만 GA의 실행 시간은 1회 실행 기준 약 366시간으로, Composite+PM의 약 0.01시간과 큰 차이가 있다.

Composite+PM과 GA는 서로를 대체하는 관계라기보다 서로 다른 역할을 가진다. Composite+PM은 계산 시간이 짧고 의사결정 기준이 명확하므로 동적 생산 환경에서 빠르고 설명 가능한 rule-based 대안으로 활용될 수 있다. 반면 GA는 반복적인 해 탐색과 시뮬레이션 평가를 통해 더 우수한 스케줄을 찾는 방식이므로, 실시간 dispatching rule보다는 충분한 계산 시간이 주어진 상황에서 해 품질을 높이기 위한 사전 최적화 또는 비교 기준으로 활용될 수 있다.

4 결론

Q-time 제약, 예방보전, 확률적 장비 고장이 동시에 작용하는 반도체 공정 스케줄링을 대상으로, 기본 dispatching rule, Composite rule, PM threshold, GA를 SimPy 기반 이산 사건 시뮬레이션에서 비교하였다. 주요 결과는 특정 방법의 절대적 우수성보다, 각 방법이 서로 다른 의사결정 범위와 계산 비용을 가진다는 점을 보여준다.

단일 dispatching rule은 해석 가능성과 빠른 계산이라는 장점에도 하나의 기준만 강조하여, Q-time·setup·waiting·전환 위험이 함께 작용하는 환경의 종합 목적함수에서는 한계를 보였다. Composite rule은 Q-time slack과 setup을 중심으로 여러 priority term을 결합해 모든 단일 rule보다 낮은 J 를 달성했고, 본 환경에서는 단일 기준보다 복수의 상태 정보를 함께 반영하는 방식이 더 적합함을 보였다.

Composite rule에 PM threshold를 결합하면 고장 횟수를 줄이면서도 PM 미적용 Composite rule과 비슷한 수준의 J 를 유지할 수 있었다. 이는 PM이 단순한 유지보수 조건이 아니라, 계획 비가동과 고장 위험 사이의 trade-off를 조정하는 스케줄링 요소임을 보여준다.

GA는 작업 순서·장비 배정·PM threshold를 함께 탐색해 Q-time violation과 처리 중 고장을 낮췄고, 평균 J 기준으로도 Composite+PM보다 근소하게 낮은 값(0.9143 vs 0.9193)을 보였다. 그러나 1회 실행에 약 366시간이 소요되어 계산 비용이 매우 컸고, 선택된 해의 의사결정 구조를 설명하기 어렵다는 한계가 있었다.

따라서 실제 운영 상황에서는 계산 시간이 제한적이고 의사결정 근거가 필요한 경우 Composite+PM이 실용적인 대안이 될 수 있다. 반면 충분한 계산 시간이 주어지고 Q-time violation 감소가 중요한 경우 GA는 사전 최적화 또는 비교 기준으로 활용될 수 있다. 즉 두 방법은 단순히 성능 수치만으로 우열을 판단하기보다, 운영 목적과 계산 가능 시간에 따라 선택되어야 한다.

4.1 연구 의의

해당 연구의 의의는 단순히 특정 알고리즘의 성능을 비교한 데 그치지 않고, 실제 반도체 제조 환경에서 동시에 발생하는 여러

운영 조건을 하나의 최적화 문제 안에 통합했다는 점에 있다. 기존 연구들이 FJSP 구조, Q-time 제약, 예방보전, 장비 고장 등을 개별적으로 다룬 경우가 많았다면, 본 연구는 Q-time violation에 따른 품질 제약, PM으로 인한 계획 비가동, 확률적 장비 고장에 따른 가용성 변화를 함께 반영하였다. 따라서 본 연구의 결과는 이상적인 정적 스케줄링 환경이 아니라, 실제 FAB 운영에 가까운 복합적이고 불확실한 조건에서 생산 효율성과 품질 제약 관리 사이의 균형을 탐색했다는 점에서 의미가 있다.

4.2 한계 및 향후 연구

본 연구의 시뮬레이션은 장비별 처리시간, setup, 고장, 수리, PM, Q-time을 반영했으나 lot 이송 시간, buffer capacity, chamber 단위 제약, batch processing, 작업자 제약, rework 정책은 포함하지 않았다. 이 요소들은 실제 FAB에서 dispatching 결과를 바꿀 수 있으므로, AMR·OHT 물류 자원과 장비 간 이동 시간, chamber-level 제약을 추가해 실제 환경에 더 가까운 검증으로 확장할 수 있다.

목적함수는 makespan과 AQT를 정규화해 결합했지만, 실제 환경에서 Q-time violation의 영향은 공정 단계·제품·초과 시간·재작업 가능 여부·scrap cost에 따라 달라진다. Q-time violation을 단순 시간 penalty가 아니라 품질 비용 또는 손실 함수로 확장하고, scrap·rework·PM·repair·납기 지연 비용을 함께 반영하면 PM threshold를 성능 지표가 아닌 경제적 기준으로 평가할 수 있다.

PM threshold는 목적함수가 가장 낮은 값을 탐색하는 방식으로 평가했고 makespan·AQT·처리 중 고장의 변화 경향까지 확인했으나, 작업 중 고장의 위험 비용, 장비별 중요도, spare capacity, 부품 교체 비용, 엔지니어 가용성 같은 PM의 경제·위험 요소는 명시적으로 모델링하지 못했다. 이들을 비용 항으로 포함하면 PM 의사결정을 더 현실적으로 다룰 수 있다.

Composite rule의 가중치와 GA의 동작은 현재 실험 조건에 맞추어 정해졌다. job mix, 장비 수, Q-time tightness, setup 구조, 고장 빈도가 달라지면 최적 가중치도 달라질 수 있고, GA 역시 population size·generation·crossover/mutation rate·fitness 가중치에 민감하다. 따라서 WIP 수준·장비 utilization·고장률에 따라 Composite weight와 PM threshold를 동적으로 조정하는 adaptive rule과 GA 파라미터의 체계적 민감도 분석이 필요하다.

5 참고문헌

- Al-Hinai, N., & ElMekkawy, T. Y. (2011). Robust and stable flexible job shop scheduling with random machine breakdowns using a hybrid genetic algorithm. *International Journal of Production Economics*, 132(2), 279–291. <https://doi.org/10.1016/j.ijpe.2011.04.020>
- Barlow, R. E., & Proschan, F. (1965). *Mathematical theory of reliability*. John Wiley & Sons.

- Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41, 157–183. <https://doi.org/10.1007/BF02023073>
- Cassady, C. R., & Kutanoglu, E. (2005). Integrating preventive maintenance planning and production scheduling for a single machine. *IEEE Transactions on Reliability*, 54(2), 304–309. <https://doi.org/10.1109/TR.2005.845967>
- Cho, L., Park, H. M., Ryan, J. K., Sharkey, T. C., Jung, C., & Pabst, D. (2014). Production scheduling with queue-time constraints: Alternative formulations. *IIE Annual Conference and Expo 2014*, 282–291.
- Kim, S., Park, S., Yoon, Y.-S., & Oh, H. (2021). A study on the scheduling of dyeing processing production based on genetic algorithm. *The Journal of Korean Institute of Communications and Information Sciences*, 46(1), 192–200. <https://doi.org/10.7840/kics.2021.46.1.192>
- Park, I. B., & Huh, J. (2026). A genetic algorithm with feasibility-agnostic encoding and three-phase decoding for scheduling semiconductor manufacturing facilities under queue time limits. *Expert Systems with Applications*, 301, 130310. <https://doi.org/10.1016/j.eswa.2025.130310>
- Pezzella, F., Morganti, G., & Ciaschetti, G. (2008). A genetic algorithm for the flexible job-shop scheduling problem. *Computers & Operations Research*, 35(10), 3202–3212. <https://doi.org/10.1016/j.cor.2007.02.014>